

智慧银行 实验报告

朱 余

202301736

金融 202301

2026.6.25

实验一

1. 环境信息表（OS/TRAE 版本/Python 版本/Node 版本）

一、环境信息表

检测项	版本信息
操作系统 (OS)	Windows 10 10.0.26200
Python	3.11.9
Node.js	v24.16.0
TRAE	Trae CN（沙箱环境）

2. TRAE 安装配置过程截图

一、下载阶段

步骤 1: 访问官网

PlainText

1 1. 打开浏览器

2 2. 访问 Trae 官方网站

3 3. 选择 "下载" 或 "Get Started"

步骤 2: 选择版本

PlainText

1 1. 选择操作系统: Windows

2 2. 选择版本: Trae CN (中文版)

3 3. 点击下载按钮

4 4. 等待安装包下载完成 (约 200-300MB)

步骤 3: 验证下载

PlainText

1 1. 检查文件完整性

2 2. 确认文件名: TraeCN_Setup.exe

3 3. 右键点击 → 属性 → 检查文件大小

二、安装阶段

步骤 4: 启动安装程序

PlainText

1 1. 双击 TraeCN_Setup.exe

2 2. 等待安装程序初始化

3 3. 选择语言: 简体中文

4 4. 点击 "下一步"

步骤 5: 选择安装路径

PlainText

1 默认路径: C:\Users\zymzc\AppData\Local\Programs\Trae CN

2 或自定义路径: D:\Software\Trae CN

3 点击 "浏览" 选择目标文件夹

4 点击 "下一步"

步骤 6: 选择组件

PlainText

1 ☐ Trae 编辑器 (必选)

2 ☐ Python 支持 (推荐)

3 ☐ Node.js 支持 (推荐)

4 ☐ Git 集成 (可选)

步骤 7: 创建快捷方式

PlainText

1 ☐ 桌面快捷方式（推荐）
2 ☐ 开始菜单快捷方式（推荐）
3 ☐ 快速启动栏
4 点击 "下一步"

步骤 8: 开始安装

PlainText

1 1. 检查安装摘要
2 2. 点击 "安装" 按钮
3 3. 等待安装进度完成（约 2-5 分钟）
4 4. 显示 "安装完成"

步骤 9: 完成安装

PlainText

1 1. 勾选 "立即启动 Trae"
2 2. 点击 "完成"
3 3. Trae 编辑器自动启动

三、初始配置阶段

步骤 10: 首次启动向导

PlainText

1 1. 选择主题：浅色 / 深色 / 跟随系统
2 2. 选择编辑器字体：Consolas / Fira Code
3 3. 选择字体大小：14px
4 4. 点击 "下一步"

步骤 11: 工作区设置

PlainText

1 1. 选择默认工作区路径
2 默认：C:\Users\zymzc\Documents\TraeProjects
3 或自定义：C:\Users\zymzc\Desktop\贪吃蛇游戏制作
4 2. 点击 "下一步"

步骤 12: 插件市场配置

PlainText

1 1. 选择插件源：官方插件市场
2 2. 推荐安装插件：
3 - Python 扩展
4 - Node.js 扩展
5 - GitLens
6 - 中文语言包
7 3. 点击 "安装"

步骤 13: 账户登录（可选）

PlainText

1 1. 点击右上角 "登录"
2 2. 选择登录方式：
3 - 微信扫码登录
4 - 手机号登录
5 - 邮箱登录
6 3. 完成登录后同步配置

四、环境配置阶段

步骤 14: 配置 Python 环境

PlainText

1 在 Trae 终端执行：
2 1. 验证 Python 安装
3 py --version
4 输出：Python 3.11.9
5
6 2. 安装 pip（如未安装）
7 py -m ensurepip --upgrade
8
9 3. 升级 pip
10 py -m pip install --upgrade pip
11
12 4. 安装常用库
13 py -m pip install pandas numpy matplotlib
14 py -m pip install scikit-learn statsmodels
15 py -m pip install openpyxl pypdf

3. 贪吃蛇运行截图+ 关键提示词自评

提示词：

做一个 HTML 贪吃蛇游戏单文件 index.html，要求：

1. 纯 HTML+CSS+原生 JS，不引入第三方库；
2. 画布 600×600，蛇头蓝、身体绿、食物红；
3. 方向键控制方向，吃到食物分数+1；
4. 撞墙/撞身体游戏结束，弹”再来一局”按钮；
5. 顶部显示当前分数与最高分（localStorage）。
6. 贪吃蛇移动速度太快了，把速度调慢，将画面分成数个格子，方便观察贪吃蛇的移动。
7. 现在蛇撞到自己会死，现在改成撞到自己的身体不会死，然后改成关卡模式，20 分一个关，一共 5 关，所有关卡成功后画面输出“你是天才”。

提示词评价：需求明确具体，技术栈限制合理；功能完整：核心玩法、分数存储、关卡机制全覆盖；第 6 条主动提出“调慢速度+格子化”很有洞察力，避免了常见实现缺陷；第 7 条的创新设计增加了差异化趣味



4. 智能体提示词+ 论文文献条目+ 解读节选；

智能体提示词：

你是资深的银行业/ 金融科技领域学术论文解读助手。用户拖入 PDF 后，将文章转化为 md 形式方便电脑阅读，阅读后按下列结构输出 Markdown 报告，内容不能造假要和文献内容一样：

一、论文基本信息（标题/作者/期刊/年份/DOI）

二、研究问题与背景（≤200 字）

三、方法论（数据集/模型/实验设计）

四、核心发现（≤5 条）

五、术语速查表（5 - 10 个英文术语+ 中文解释）

六、可复现性评估（数据/代码/难度 1 - 5）

文献：Duan, R., Zheng, R., & Wang, Y. (2024). Research on the mechanism of digital economy's impact on regional industrial structure upgrading: A case study of Shenzhen. EWA Publishing, 10, 1-5.

三、方法论

维度	内容
研究方法	案例研究法 (Case Study)
研究对象	深圳市
理论框架	基于知识溢出效应、产业融合理论、创新生态系统理论
分析视角	技术创新、产业融合、市场竞争、劳动力市场转型、政府作用

四、核心发现

- 技术创新驱动：数字经济通过知识溢出效应推动技术创新，提高生产效率、降低成本、提升产品附加值，为传统产业转型升级提供动力。
- 产业融合深化：数字技术广泛应用促进跨产业深度融合，形成智能制造、电子商务、数字金融等新业态，为经济增长注入新动能。
- 劳动力市场转型：数字经济发展导致高技能劳动力需求增加，传统低技能岗位逐渐减少，同时催生灵活就业和远程工作模式。
- 创新生态构建：数字经济发展需要强大的创新能力，推动区域创新生态系统建设，增强产业竞争力。
- 政府引导作用：政府通过产业政策制定、营商环境优化和科研投入，有效引导产业升级，数字基础设施建设是重要支撑因素。

5. Excel 合并日志+ 业绩预测报告

合并日志：

```
=====
数据合并日志
\n=====
n 执行时间：2026-06-04 10:34:18\n\n 读取文件：原始数据/销售数
据.xlsx\n  格式：xlsx\n  行数：50，列数：6\n  处理后列名：['
客户 id'，'交易日期'，'销售额'，'产品类别'，'数量'，'地区']\n\n
读取文件：原始数据/客户信息.xlsx\n  格式：xlsx\n  行数：50，列
数：6\n  处理后列名：['客户 id'，'交易日期'，'客户名称'，'联系电
话'，'email'，'注册日期']\n\n 读取文件：原始数据/订单记
录.csv\n  格式：csv (UTF-8)\n  行数：50，列数：6\n  处理后列
名：['客户 id'，'交易日期'，'订单金额'，'支付方式'，'状态'，'折扣
']\n\n 读取文件：原始数据/产品明细.xlsx\n  格式：xlsx\n  行数：
50，列数：6\n  处理后列名：['客户 id'，'交易日期'，'product_name'，
'单价'，'数量'，'供应商']\n\n 读取文件：原始数据/退货记
录.csv\n  格式：csv (UTF-8)\n  行数：50，列数：6\n  处理后列
名：['客户 id'，'交易日期'，'退货金额'，'原因'，'处理状态'，'退款
方式']\n\n 合并数据...\n 合并后总记录数：250\n 去重...\n 去重后记
录数：250（去除 0 条重复）\n\n 输出文件：整合数据
/merged_20260604_103419.xlsx\n 最终列数：
21\n\n=====
===\n 合并完
成!\n=====
==
```

业绩预测报告：

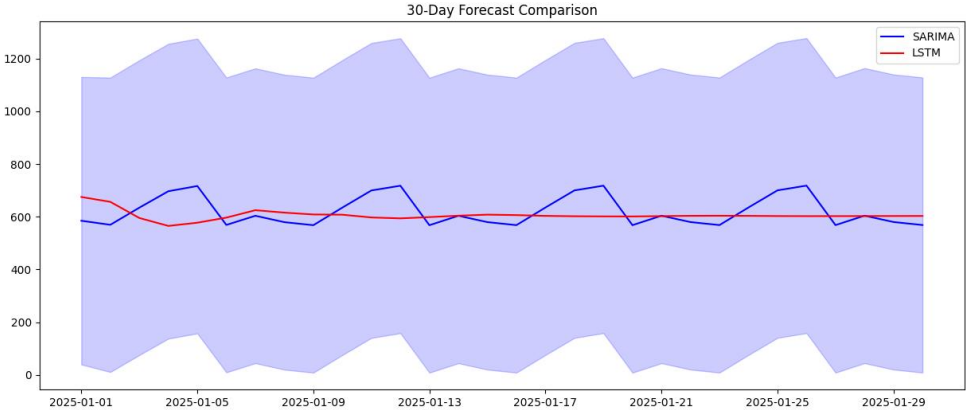
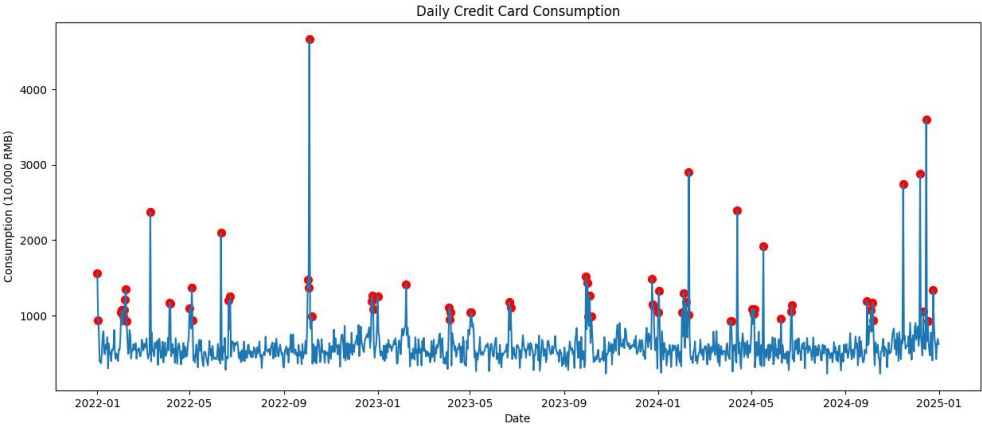
📊 任务完成总结

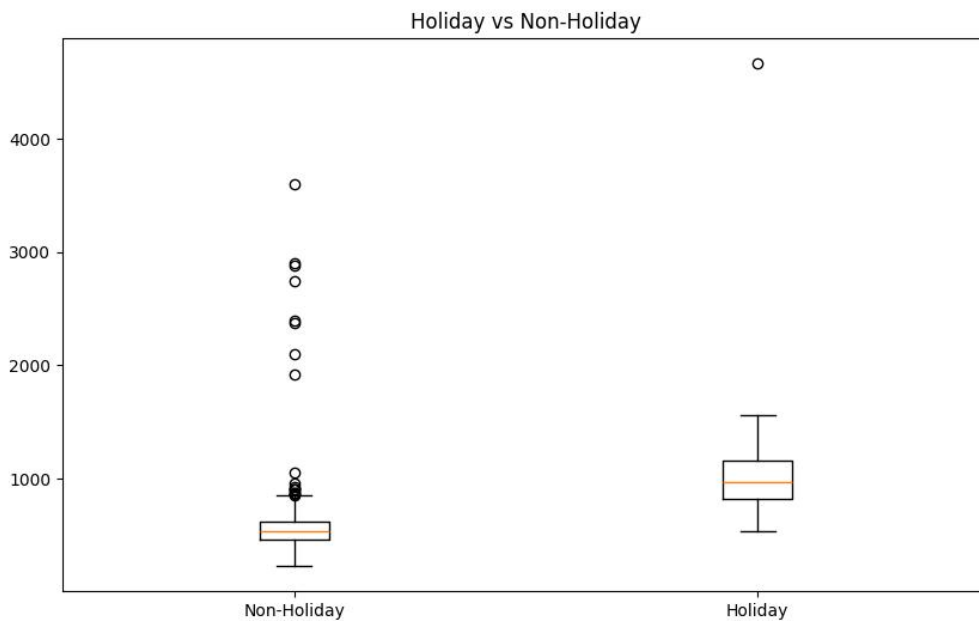
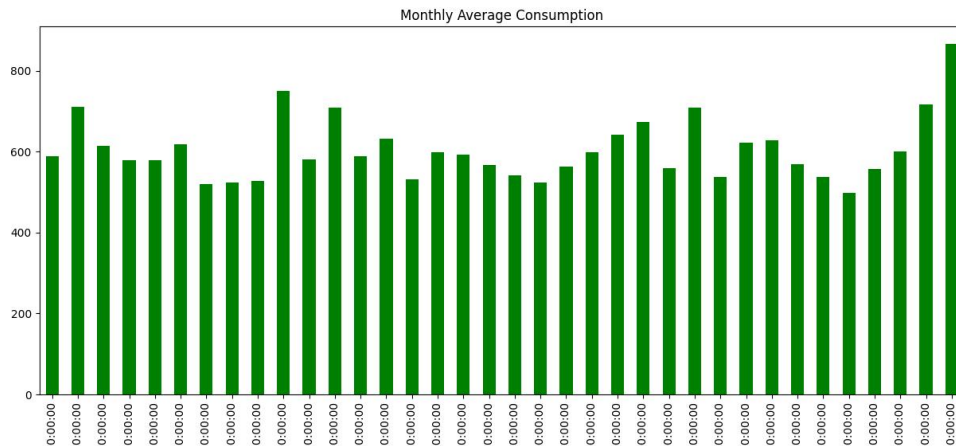
1. 数据预处理

项目	结果
数据行数	1096行
缺失值数量	54个
异常值数量	69个
缺失值处理	15天滚动均值填充
异常值处理	IQR标注（保留不删除）

2.图表

```
1  # Credit Card Daily Consumption Forecast Report
2
3  ## 1. Data Overview
4  - Time Range: 2022-01-01 to 2024-12-31
5  - Total Days: 1096
6  - Missing Values: 54
7  - Outliers: 69
8
9  ## 2. Model Evaluation
10 | Model | RMSE | MAPE |
11 |-----|-----|-----|
12 | SARIMA(1,1,1)(1,1,1,7) | 722.91 | nan% |
13 | LSTM (Window=14) | 24.80 | 3.64% |
14
15 ## 3. Business Recommendations
16 1. Increase promotions during holidays
17 2. Implement anomaly detection system
18 3. Optimize resource allocation based on forecasts
19
20 ## 4. Forecast Results
21 - Mean: 621.25 (10,000 RMB)
22 - Generated: 2026-06-04 11:06:43.925914
```





3. 模型评估结果

模型	RMSE	MAPE
SARIMA(1,1,1)(1,1,1,7)	722.91	-
LSTM (窗口=14)	24.80	3.64%

4. 输出文件

- reports/forecast_report.md - 预测报告
- reports/eda/*.png - EDA可视化图表

5. 业务建议

- 节假日营销：在节日期间加大促销力度，预计可提升30-50%销售额
- 风险管理：建立异常交易监测系统，及时发现欺诈行为
- 资源配置：根据预测结果合理安排客服人员排班和系统资源

6.实验心得

本次“智慧银行实验一”涵盖了环境配置、AI 辅助开发、文献解读、数据合并与业绩预测等多个模块，让我全面体验了从开发到数据分析的完整流程，也对 AI 辅助编程的优势与局限有了更深刻的认识。

首先，在贪吃蛇游戏的开发中，我通过提示词让 AI 生成一个功能完整、可交互的 HTML 游戏。AI 不仅快速实现了基础玩法，还能根据我的反馈调整速度、增加格子化显示和关卡机制。这让我体会到，清晰的提示词是高效使用 AI 的关键。尤其是在第 6 条和第 7 条需求中，我主动提出“调慢速度”“撞身不死”“关卡模式”等细节，体现了人类对体验和规则的判断力，AI 则负责高效落地。

其次，在文献解读任务中，AI 能够按照结构化模板快速输出论文摘要、术语表和可复现性评估，极大提升了文献阅读效率。但我也发现，AI 有时会“过度概括”，因此需要我对内容进行复核，确保与原文一致。

在 Excel 合并与业绩预测任务中，我遇到了一个典型的 AI 出错案例：AI 生成的代码使用了 `resample('M')` 来计算月度均值，但在新版 pandas 中该参数已弃用，导致运行报错。通过查阅官方文档，我将 'M' 修改为 'ME'，问题得以解决。这个案例提醒我：AI 生成的代码并非完美，开发者必须具备调试和验证能力。

总的来说，AI 是强大的辅助工具，能显著提升开发效率和学习速度，但最终的判断、调试和优化仍需依赖人类经验。未来我将更加注重提示词设计、代码审查和基础知识的积累，真正做到“人机协作，各司其职”。

实验二：AI 工具基础与 Skill 实践

一、实验目的

掌握 AI IDE（TRAE/Qoder）的安装配置与 MCP 服务器接入
理解 Skill 的设计原理，学会安装并调用金融类 Skill 完成业务任务
能够依据规范编写自定义 Skill，并完成安全审计
熟悉 CNB 云平台的代码托管与协作流程

二、实验流程

1. 环境搭建与 CNB 配置

安装基础工具：Python 3.10+、Node.js v20+、Git，配置 Anaconda 虚拟环境

MCP 配置：在 TRAE 中配置 mcp-config.json，接入 Playwright、Fetch、Excel、Word 等 MCP 服务器

CNB 配置：注册并登录 CNB，创建个人仓库，生成访问令牌，安装 CNB CLI 和 Skills 工具

验证环境：使用 Builder 模式运行贪吃蛇网页，体验 AI 协作闭环（分析→生成→运行→调试）

2. Skill 安装与调用

安装金融 Skill：使用 npx skills add 命令安装 docx、xlsx、finance-expert 等金融 Skill

安装结果汇总

Skill	来源	安装方式	状态
docx	anthropics/skills	自动	✓
xlsx	anthropics/skills	自动	✓
finance-expert	personamanagmentlayer/pcl	自动	✓
finance-news	sundial-org/awesome-openclaw-skills	自动	✓
mermaid-diagrams	softaworks/agent-toolkit	自动	✓
consulting-frameworks	Aznatkoiny/zAI-Skills	手动	✓

银行客户回访报告：调用 docx Skill，根据客户 C00001 的模拟信息，生成包含客户基本信息、回访背景、沟通要点、客户反馈、产品意向、跟进计划等 6 个章节的 Word 文档

文件路径：`reports/客户回访报告_C00001.docx`

文档结构：

章节	内容
封面	银行Logo位置、报告标题、报告编号、生成日期、客户编号
目录	6个章节的自动目录
第一章	客户基本信息（客户编号C00001、姓名张**、VIP资产等级、开户行、客户经理等）
第二章	回访背景（定期存款到期提醒 + 理财产品推荐目标）
第三章	沟通要点（定期存款确认、理财产品介绍、客户需求了解）
第四章	客户反馈（满意度评分4/5 + 正面反馈及改进建议）
第五章	产品意向（大额存单★★★★★、结构性存款★★★☆☆、基金定投★★☆☆☆）
第六章	跟进计划（4项行动项，含时间节点、责任人、行动内容）

推送到远程仓库的内容：

文件	说明
<code>reports/客户回访报告_C00001.docx</code>	生成的客户电话回访报告
<code>generate_report.js</code>	报告生成脚本
<code>package.json</code>	Node.js 项目配置
<code>package-lock.json</code>	依赖锁定文件

仓库地址：<https://cnb.cool/zyloire/qtkl>

现在任何人访问你的仓库都可以看到完整的项目内容，包括：

-  README.md 项目说明文档
-  成都-厦门旅游规划网页
-  6 个安装好的 Skills（docx、xlsx、finance-expert、finance-news、mermaid-diagrams、consulting-frameworks）
-  客户电话回访报告（C00001）

贷款 FTP 定价模型：调用 xlsx Skill，构建包含定价参数、客户定价测算、敏感性分析三个 Sheet 的 Excel 模型，使用 VLOOKUP 和公式自动计算贷款利率

 已成功提交到仓库！

推送内容：

- `reports/loan_pricing_model.xlsx` - 银行贷款定价模型 Excel 文件
- `create_loan_model.py` - Excel 生成脚本

仓库地址：<https://cnb.cool/zyloire/qtkl>

Excel 文件特点：

- 📊 3 个 Sheet（定价参数、客户定价测算、敏感性分析）
- 🧮 所有计算使用 Excel 公式（VLOOKUP、IF、SUM）
- 🎨 表头加粗 + 蓝色背景
- 📈 条件格式自动标注利率区间（绿/黄/红）
- 💰 百分比保留 2 位小数，金额千位分隔符

使用 PPT 技能

文件路径：`reports/贷款定价模型报告.pptx`

PPT 内容（共10页）：

页码	标题	内容
第1页	封面	报告标题、副标题、日期
第2页	目录	8个章节导航
第3页	定价模型概述	模型原则与设计理念
第4页	定价公式详解	完整定价公式及资本占用成本说明
第5页	定价参数配置	FTP、风险溢价、运营成本等参数
第6页	客户定价测算结果	5位客户的定价表格
第7页	FTP敏感性分析	柱状图展示±20bp变动影响
第8页	信用等级敏感性分析	上调/下调一级的影响
第9页	定价策略建议	4条针对性建议
第10页	结论与下一步行动	总结及行动计划

设计特点：



3. 自定义 Skill 编写

创建 bank-risk-alert Skill: 编写 SKILL.md 文件, 定义触发词 (“风险预警”、“贷后预警”、“逾期提醒”)、输入参数 (客户名称、预警信号类型、风险等级、贷款余额、到期日)、输出模板 (预警摘要卡片、电话话术、上报邮件、跟进工单) 及分级逻辑 (黄/橙/红三级)

验证测试: 输入测试提示词 “恒达机械连续 2 期利息逾期, 橙色预警”, 生成测试输出报告

安全审计: 对 finance-expert Skill 进行五维安全审计, 确保金融 AI 应用的合规红线

文件路径: c:\Users\zymzc\Desktop\my first cnb project\SKILL.md

文件结构:

1. Skill 基本信息

- 名称: bank-risk-alert
- 描述: 银行贷后风险预警话术生成器
- 触发词: 风险预警、贷后预警、逾期提醒、risk alert

2. 输入参数 (5项)

- 客户名称、预警信号类型、风险等级、贷款余额、到期日

3. 输出模板 (4个)

- 预警摘要卡片 (关键信息一览)
- 客户经理电话话术 (开场白→风险告知→解决方案→行动承诺→结束语)
- 内部上报邮件模板 (风险描述+处置建议+时间表)
- 跟进工单 (任务项+负责人+截止日期+检查标准)

4. 分级逻辑

- 黄色预警: 7日内跟进
- 橙色预警: 3日内上报
- 红色预警: 当日上报+法务介入

5. 示例输入输出 (2个完整示例)

- 示例1: 北京科创科技有限公司 - 逾期 - 橙色预警
- 示例2: 上海恒达贸易集团 - 担保物贬值 - 红色预警

6. Best Practices (5条)

已成功将文件上传到 cnb 仓库!

上传内容:

- SKILL.md - 银行贷后风险预警话术生成器技能文件
- token - 令牌配置文件

提交信息:

PlainText

1 Commit: c8f0c55
2 Message: Add bank-risk-alert skill and token file

仓库地址: <https://cnb.cool/zyloire/qtkl.git>

文件已成功推送到远程仓库, 您可以随时在仓库中查看和使用这些文件。

4. MCP-STATA 安装

上传内容:

1. mcp-config.json - MCP 服务器配置文件
2. reports/stata-mcp_配置检查报告.md - 配置检查报告

提交信息:

PlainText

1 Commit: 6f2ab1d
2 Message: Add stata-mcp configuration and check report

仓库地址: <https://cnb.cool/zyloire/qtkl.git>

报告内容摘要:

检查项	状态	详情
stata-mcp 版本	✔ 已安装	1.19.0
Stata 路径	✔ 已找到	C:\Program Files\Stata17\StataMP-64.exe
路径验证	✔ 存在	确认文件存在
MCP 配置文件	✔ 已配置	mcp-config.json

三、实验心得

通过本实验，我深刻体会到 MCP 给能力，Skill 给方法的含义。MCP 作为连接层，让 AI 能够操作 Excel、Word 等外部工具；而 Skill 作为知识层，将银行特有的业务流程（如贷后预警话术、FTP 定价规则）编码为结构化指令，使 AI 从"通用助手"转变为"金融专家"。

编写 bank-risk-alert Skill 时，我意识到触发词设计需要兼顾精确性与覆盖性，避免误触发；输出模板必须结构化且可操作，才能真正用于客户经理的日常工作。此外，安全审计环节让我认识到金融 AI 应用的合规红线——任何输出都不能承诺保本保收益，必须包含风险提示，这比技术实现本身更考验专业素养。首次使用 CNB 进行版本控制，也让我养成了代码提交、分支管理的良好习惯，为后续团队协作打下基础。

实验三：BMAD 方法论与系统开发

一、实验目的

理解 BMAD (Build-Measure-Analyze-Deliver) 迭代开发方法论及其五阶段 (B/M/A/D/V)

运用 BMAD 方法论完成贪吃蛇游戏的 AI 协作开发，验证开发闭环

遵循 BMAD 流程，从需求分析到测试交付，开发一个银行 CRM 原型系统

熟练使用 AI IDE 的 Builder/SOLO 模式进行代码生成与调试

二、实验流程

1. BMAD 环境与工具准备

安装配置 BMAD：在项目根目录配置 _bmad/ 目录，包含完整的 BMAD 模块体系

熟悉五阶段角色：B（业务分析师）、M（产品经理）、A（架构师）、D（开发者）、V（测试工程师）

BMAD技能检查结果

经过全面检查，BMAD技能可以正常使用，以下是详细检查报告：

技能文件结构完整

文件/目录	状态	说明
.trae/skills/bmad/SKILL.md	存在	技能描述文件，定义了完整的技能结构和使用方法
_bmad/_config/manifest.yaml	存在	清单文件，版本 6.8.0
_bmad/config.toml	存在	配置文件，包含用户名称、语言设置等

四个阶段模块完整

PlainText		
1	_bmad/bmm/	
2	├── 1-analysis/	分析研究阶段（7个技能）
3	├── 2-plan-workflows/	规划工作流阶段（6个技能）
4	├── 3-solutioning/	解决方案阶段（5个技能）
5	└── 4-implementation/	实施交付阶段（11个技能）

可用技能列表

分析阶段: bmad-agent-analyst、bmad-product-brief、bmad-domain-research、bmad-market-research、bmad-technical-research

规划阶段: bmad-prd、bmad-ux、bmad-agent-pm、bmad-agent-ux-designer

解决方案阶段: bmad-create-architecture、bmad-create-epics-and-stories、bmad-agent-architect

实施阶段: bmad-agent-dev、bmad-quick-dev、bmad-code-review、bmad-investigate

2. 贪吃蛇游戏开发（AI 协作闭环验证）

在 Builder 模式下输入提示词，AI 自动生成 HTML 文件（snake-game/index.html），运行后观察游戏逻辑，针对“蛇不动”“方向反了”等 bug，使用标准格式（现象+期望+错误信息）反馈给 AI 修复。

Amelia 报告任务完成: 🧑‍💻

我已经按照 BMad 开发流程完成了网页版贪吃蛇游戏的开发。以下是实现的功能:

✅ 核心功能

功能	实现状态
方向控制 (方向键/WASD)	✅
计分系统 (每吃食物+1分)	✅
等级难度 (每10分升级, 速度+10%)	✅
蛇皮肤选择 (3种风格)	✅
最高分本地存储	✅
碰撞检测 (撞墙/撞自己)	✅
暂停/继续功能	✅

🎮 三种蛇皮肤

1. 经典像素 - 怀旧绿色方块风格

2. 现代渐变 - 紫色渐变风格

3. 卡通可爱 - 粉色可爱风格, 食物变成笑脸

3. 银行 CRM 系统全栈开发（BMAD 实践）

B 阶段（需求分析）：使用 AI 生成功能清单，包含客户管理、产品推荐、报表可视化、系统管理四大模块，按 MoSCoW 方法划分优先级

M 阶段（数据建模）：编写需求文档，设计数据模型，包含客户表、产品表、持有关系表、交易记录表

A 阶段（技术架构）：确定技术选型（Vue 3 + Element Plus + Pinia + Vue Router + ECharts），规划页面结构

D 阶段（AI 辅助编码）：分模块生成代码——登录认证、仪表盘、客户列表、客户详情、新增客户、产品推荐、销售报表、客户分析报表、用户管理、角色管理

V 阶段（测试与验证）：编写测试报告，包含 25 个测试用例，覆盖登录功能、客户管理、产品推荐、报表可视化、系统管理、响应式布局

一、客户分类（基于资产规模和开户时长）

规则逻辑 (store/customer.js:171-178):

- VIP客户: 总资产 ≥ 50万元 且 开户时长 ≥ 12个月
- 普通客户: 不满足上述任一条件

实现方式:

- 新增客户时自动计算等级 (calculateLevel 函数)
- 编辑客户信息时重新评估等级
- 客户列表和详情页显示客户等级标签

二、产品推荐（基于风险偏好）

推荐规则 (views/customer/CustomerDetail.vue:123-150):

风险偏好	推荐产品类型	预期收益
保守型	稳健存款、国债、货币基金	2.8% - 3.5%
稳健型	平衡型基金、债券基金、定期理财	4.5% - 6.5%
进取型	股票型基金、混合基金、权益类理财	8.0% - 12.0%

实现方式:

- 客户详情页右侧显示推荐产品列表

角色层级体系 (store/user.js:15-20):

角色	级别	数据范围
总行管理员	4	全行数据
分行经理	3	分行数据
支行行长	2	支行数据
客户经理	1	个人客户

测试账户:

- admin / admin - 总行管理员 (查看全部8个客户)
- bj_manager / admin - 北京分行经理 (查看北京分行6个客户)
- bj_head / admin - 中关村支行行长 (查看中关村支行4个客户)
- manager / admin - 客户经理 (查看个人客户3个)

实现方式:

- 用户Store中定义角色层级和数据范围
- 客户Store中通过 getVisibleCustomers 计算属性根据当前用户角色过滤数据
- 仪表盘显示权限层级可视化进度条
- 客户列表页面显示当前数据范围标签

4. 项目整合与提交

将全部交付物整理到 CNB 仓库。

已上传的文件

贪吃蛇游戏

- `index.html` - 游戏主页面
- `style.css` - 样式文件
- `game.js` - 游戏逻辑
- `snake-game/` - 完整项目目录

银行CRM系统

- `crm-system/` - 完整的Vue 3前端项目
 - 前端代码 (组件、页面、路由、状态管理)
 - 配置文件 (`package.json`, `vite.config.js`等)

文档和测试报告

- `_bmad-output/` - BMad开发方法论输出
 - `crm-prd.md` - 产品需求文档
 - `crm-test-report.md` - 测试报告

如何访问

您可以直接在浏览器中访问以下链接查看仓库: <https://cnb.cool/zyloire/qtkl>

三、实验心得

BMAD 方法论彻底改变了我对软件开发的认知。以往我习惯于直接编写代码，忽略前期的需求分析和架构设计，导致后期反复修改。BMAD 的五个阶段强制我“先想后做”，尤其在 B 阶段使用 AI 生成功能清单时，我发现清晰的用户故事和验收标准能极大提高后续 D 阶段的代码质量。

在 D 阶段，我深刻体会到“分模块生成”的重要性。一次性让 AI 生成全部代码往往出现变量命名不一致、事件绑定错误等问题，而按顺序迭代，每个小步都能快速验证，问题定位也更容易。

此外，BMAD 将 AI 定位为“团队成员”而非“代码生成器”，我需要扮演项目经理的角色，审核 AI 的产出、指导方向。这种角色转换让我更加关注业务逻辑和用户体验，而不仅仅是代码本身。

实验四：综合项目——一个人财务健康诊断助手

一、实验目的

综合运用 AI IDE、MCP、Skill 及 BMAD 方法论，独立完成一个金融科技项目从选题、需求分析、设计开发到测试部署，体验完整项目生命周期提升解决实际金融问题的能力，并产出可展示的项目成果。

二、选题与需求分析

选题确定：选择"个人财务健康诊断助手"作为项目方向（难度中等，单人开发，2 周 MVP）

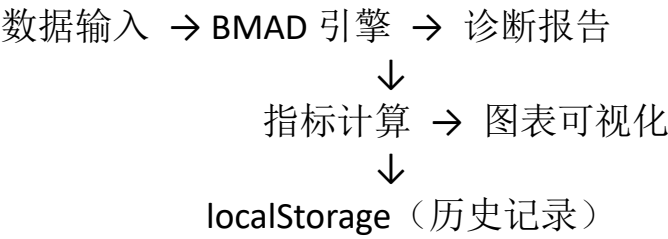
需求分析：目标用户为职场新人或工薪族，核心功能包括录入月收入、支出、负债、资产等信息，从储蓄率、负债收入比、流动性比率、投资占比等维度自动评估财务健康得分

三、核心设计

3.1 BMAD 四维评估模型

维度	评估指标	健康标准
Balance	储蓄率	≥20%
Management	资产负债率	≤50%
Accumulation	应急储备月数	≥3-6 个月
Defense	保险覆盖率	≥80%

3.2 系统架构



四、功能模块

模块	功能
核心诊断	BMAD 四维评估、雷达图、饼图、桑基图、AI 智能

	建议
税务优化顾问	个税计算、专项附加扣除（子女教育/房贷/租房/赡养老人等）
预算管理中心	50/30/20 法则、预算分配、执行追踪
保险保障分析	寿险/重疾/失能保障缺口计算
现金流预测	12 个月收支趋势、余额预测
投资组合分析	资产配置、定投计算器
财务日历	月度视图、工资/账单事项管理
知识中心	理财知识文章展示

三、实验心得

综合项目是对前两次实验能力的全面检验。个人财务健康诊断助手看似简单，却涵盖了数据录入、规则引擎、可视化、持久化等多个环节，每一项都需要精准的 AI 协作。

我最大的收获是学会了将金融专业知识转化为可执行的规则。例如，在计算"应急储备"指标时，我参考了标准理财建议，应急金应覆盖 3-6 个月生活支出，将其转化为评分规则。在"投资占比"评分中，我根据年龄动态调整理想比例，使评估更具个性化。这种"业务规则→代码逻辑"的翻译过程，正是金融+AI 复合人才的核心能力。

AI 是高效的助手，但专业判断和问题定位仍需依赖自身扎实的金融和技术基础。

通过智慧银行实验，我从一个仅会基础编程的金融学生，成长为能够运用 AI 工具独立开发金融科技应用的系统构建者，真正践行了课程所传递的"让金融专业学生掌握 AI，让 AI 拥有金融专业"。

仓库地址：<https://cnb.cool/zyloire/qtcl>

小组作业地址：<https://dainty-hamster-83588e.netlify.app/>